

UNIT 2 DATA PRE PROCESSING

Why data preprocessing - Steps of preprocessing - Data cleaning – Missing values, Noisy data, Removing outliers - Data integration – Data transformation – Generalization - Normalization - Min-max normalization, Z-Score normalization, Decimal scaling normalization, Decimal scaling normalization, Attribute selection, Label encoding – One Hot encoding- Data reduction - Dimensionality reduction – PCA.

Why data preprocessing?

Data preprocessing is the process of preparing raw data for analysis by cleaning and transforming it into a usable format. In data mining it refers to preparing raw data for mining by performing tasks like cleaning, transforming, and organizing it into a format suitable for mining algorithms.

Enhances Data Quality: Eliminates inconsistencies, redundancies, and errors

Improves Model Performance: Promotes better accuracy and efficiency in ML algorithms

Reduces Computational Complexity: Optimizes data structures for faster processing

Facilitates Better Insights: Leads to more reliable business intelligence and decision-making

In real world the available data is

1. Incomplete data
2. Inaccurate data

3. Outlier data
4. Data with missing values
5. Data with inconsistent values
6. Duplicate data

Data preprocessing improves the quality of the data mining techniques. The raw data must be preprocessed to give accurate results. The process of detection and removal of errors in data is called data cleaning. Data wrangling means making the data processable for machine learning algorithms. Some of the data errors include human errors such as typographical errors or incorrect measurement and structural errors like improper data formats. Data errors can also arise from omission and duplication of attributes. Noise is a random component and involves distortion of a value or introduction of spurious objects. Often, the noise is used if the data is a spatial or temporal component. Certain deterministic distortions in the form of a streak are known as artifacts.

Data preprocessing involves the following steps:

Data Cleaning: Identifying and fixing errors, handling missing values, and removing duplicates

Data Transformation: Standardizing, normalizing, or encoding categorical data to improve model compatibility

Data Integration: Merging data from multiple sources into a unified dataset

Feature Scaling: Adjusting numerical values to promote fair weightage in ML models

Dimensionality Reduction: Eliminating irrelevant or redundant features to

improve efficiency

Data Splitting: Dividing data into training, validation, and testing sets for ML model evaluation

Data Cleaning:

Data cleaning is a step in machine learning (ML) which involves identifying and removing any missing, duplicate or irrelevant data.

- Raw data (log file, transactions, audio /video recordings, etc) is often noisy, incomplete and inconsistent which can negatively impact the accuracy of model.
- The goal of data cleaning is to ensure that the data is accurate, consistent and free of errors.
- Clean datasets also important in [EDA](#) (Exploratory Data Analysis) which enhances the interpretability of data so that the right actions can be taken based on insights.

Common Data Cleaning Tasks

Data cleaning involves several key tasks, each aimed at addressing specific issues within a dataset. Here are some of the most common tasks involved in data cleaning:

1. Handling Missing Data

Missing data is a common problem in datasets. Strategies to handle missing data include:

- **Removing Records:** Deleting rows with missing values if they are relatively few and insignificant.
- **Imputing Values:** Replacing missing values with estimated ones, such as the mean, median, or mode of the dataset.
- **Using Algorithms:** Employing advanced techniques like regression or machine learning models to predict and fill in missing values.

2) **Noisy Data** : Noisy data are the data that cannot be interpreted by machine and are containing unnecessary faulty data. Some ways to handle them are -

- **Binning** - This method handle noisy data to make it smooth. Data gets divided equally and stored in form of bins and then methods are applied to smoothing or completing the tasks. The methods are Smoothing by a bin mean method(bin values are replaced by mean values), Smoothing by bin median(bin values are replaced by median values) and Smoothing by bin boundary(minimum/maximum bin values are taken and replaced by closest boundary values).
- **Regression** - Regression functions are used to smoothen the data. Regression can be linear(consists of one independent variable) or multiple(consists of multiple independent variables).
- **Clustering** - It is used for grouping the similar data in clusters and is used for finding outliers.

3. Removing Duplicates

Duplicates can skew analyses and lead to inaccurate results. Identifying and removing duplicate records ensures that each data point is unique and accurately represented.

4. Correcting Inaccuracies

Data entry errors, such as typos or incorrect values, need to be identified and corrected. This can involve cross-referencing with other data sources or using validation rules to ensure data accuracy.

5. Standardizing Formats

Data may be entered in various formats, making it difficult to analyze. Standardizing formats, such as dates, addresses, and phone [numbers](#), ensures consistency and makes the data easier to work with.

6. Dealing with Outliers

Outliers can distort analyses and lead to misleading results. Identifying and addressing outliers, either by removing them or transforming the data, helps maintain the integrity of the dataset.

Example:

Step1: Assess Data Quality

The first step in data cleaning is to assess the quality of your data. This involves checking for:

- Missing Values
- Incorrect Values
- Inconsistencies in Data Format

Imperfect DataFrame

Name	Age	Score	Date
John	25	90	2022-01-01
Mary	31	80	2022-01-02
Jane	22	70	2022-01-03
Bob	35	95	2022-01-04
Alice	28	85	2022-01-05
John	25	90	2022-01-01
Mary	31	80	2022-01-02
nan	40	100	2022-01-06

The faults in the DataFrame are as follows:

1. Duplicate Rows: Rows 1 and 6 are duplicates, indicating a potential data duplication issue.
2. Missing Values: Row 7 has a missing value in the "Name" column, which could affect analysis and interpretation.
3. Inconsistent Date Format: The "Date" column contains dates in the format "YYYY-MM-DD", which is consistent, but it's important to ensure consistency across all date entries.
4. Possible Outlier: The score of 100 in row 7 could be considered as an outlier, depending on the context of the data and the scoring system used.

Step2: Remove Irrelevant Data

Duplicate records can skew analysis results and lead to incorrect conclusions. Deduplication involves:

- Identifying Duplicate Entries
- Removing Duplicate Records
- Identifying Redundant Observations
- Eliminating Irrelevant Information

Imperfect DataFrame

Name	Age	Score	Date
John	25	90	2022-01-01
Mary	31	80	2022-01-02
Jane	22	70	2022-01-03
Bob	35	95	2022-01-04
Alice	28	85	2022-01-05
John	25	90	2022-01-01
Mary	31	80	2022-01-02
nan	40	100	2022-01-06

Deduplicated DataFrame

Name	Age	Score	Date
John	25	90	2022-01-01
Mary	31	80	2022-01-02
Jane	22	70	2022-01-03
Bob	35	95	2022-01-04
Alice	28	85	2022-01-05
nan	40	100	2022-01-06

In the deduplicated [DataFrame](#) Rows 1 and 6, which were duplicates, have been removed from the DataFrame.

Step 3: Handle Missing Data

Imperfect DataFrame

Name	Age	Score	Date
John	25	90	2022-01-01
Mary	31	80	2022-01-02
Jane	22	70	2022-01-03
Bob	35	95	2022-01-04
Alice	28	85	2022-01-05
John	25	90	2022-01-01
Mary	31	80	2022-01-02
nan	40	100	2022-01-06

DataFrame with Handled Missing Values

Name	Age	Score	Date
John	25	90	2022-01-01 00:00:00
Mary	31	80	2022-01-02 00:00:00
Jane	22	70	2022-01-03 00:00:00
Bob	35	95	2022-01-04 00:00:00
Alice	28	85	2022-01-05 00:00:00
Unknown	40	100	2022-01-06 00:00:00

Missing Value Handled: The missing value in the "Name" column (row 7) has been replaced with "Unknown" to signify that the name is unknown or not available. This helps to maintain data integrity and completeness.

Step 4: Normalize Data

Data [normalization](#) involves organizing data to reduce redundancy and improve storage efficiency. This typically involves:

- Splitting Data into Multiple Tables: Divide the data into separate tables, each storing specific types of information.
- Ensuring Data Consistency: Verify that data is structured in a way that facilitates efficient querying and analysis.

Normalized Data (Student Information)

Name	Age	Date
John	25	2022-01-01 00:00:00
Mary	31	2022-01-02 00:00:00
Jane	22	2022-01-03 00:00:00
Bob	35	2022-01-04 00:00:00
Alice	28	2022-01-05 00:00:00
Unknown	40	2022-01-06 00:00:00

Normalized Data (Scores)

Name	Score
John	90
Mary	80
Jane	70
Bob	95
Alice	85
Unknown	100

Step 5: Identify and Manage Outliers

Imperfect DataFrame

Name	Age	Score	Date
John	25	90	2022-01-01
Mary	31	80	2022-01-02
Jane	22	70	2022-01-03
Bob	35	95	2022-01-04
Alice	28	85	2022-01-05
John	25	90	2022-01-01
Mary	31	80	2022-01-02
nan	40	100	2022-01-06

DataFrame with Managed Outliers

Name	Score
John	90
Mary	80
Jane	70
Bob	95
Alice	85
Unknown	100

Data Integration: It involves merging data from various sources into a single, unified dataset. It can be challenging due to differences in data formats, structures, and meanings. Techniques like record linkage and data fusion help in combining data efficiently, ensuring consistency and accuracy.

Record Linkage is the process of identifying and matching records from different datasets that refer to the same entity, even if they are represented differently. It helps in combining data from various sources by finding corresponding records based on common identifiers or attributes.

Data Fusion involves combining data from multiple sources to create a more comprehensive and accurate dataset. It integrates information that may be inconsistent or incomplete from different sources, ensuring a unified and richer dataset for analysis.

Data Transformation :

The change made in the format or the structure of the data is called data transformation. This step can be simple or complex based on the requirements. There are some methods in data transformation.

a) Smoothing: With the help of algorithms, we can remove noise from the dataset and helps in knowing the important features of the dataset. By smoothing we can find even a simple change that helps in prediction.

b) Aggregation: In this method, the data is stored and presented in the form of a summary. The data set which is from multiple sources is integrated into with data analysis description. This is an important step since the accuracy of the data depends on the quantity and quality of the data. When the quality and the quantity of the data are good the results are more relevant.

c) Discretization: The continuous data here is split into intervals. Discretization reduces the data size. For example, rather than specifying the class time, we can set an interval like (3 pm-5 pm, 6 pm-8 pm).

d) Normalization: It is the method of scaling the data so that it can be represented in a smaller range. Example ranging from -1.0 to 1.0.

e) Attribute Selection: To help the mining process, new attributes are derived from the given attributes.

f) Concept Hierarchy Generation: In this, the attributes are changed from lower level to higher level in hierarchy

Applications of Data Preprocessing

Machine Learning and AI: Essential for building accurate and efficient models

Business Intelligence: Promotes clean and structured data for analytics and reporting

Healthcare Analytics: Prepares medical data for predictive analysis and AI-driven diagnostics

Financial Forecasting: Enhances data reliability for fraud detection and risk assessment

Natural Language Processing (NLP): Cleans and tokenizes text data for sentiment analysis and chatbots

Benefits of Data Preprocessing

Increases Data Usability: Transforms raw, unstructured data into actionable insights

Reduces Model Bias: Promotes fair representation and eliminates skewed datasets

Improves Efficiency: Streamlines data pipelines and speeds up analysis

Enhances Interpretability: Makes complex datasets easier to understand and visualize

Challenges in Data Preprocessing

Handling Missing Data: Requires strategies like imputation or removal to maintain dataset integrity

Balancing Data: Avoiding class imbalances in classification problems

Scaling Large Datasets: Efficiently managing extensive, high-dimensional data

Data Privacy Concerns: Complying with data protection regulations like GDPR and CCPA

Generalization

Generalization in machine learning refers to a model's ability to apply learned patterns to new, unseen data. It is a key factor in predictive modeling, ensuring that models perform well in real-world situations rather than just memorizing training examples. When a model generalizes well, it can accurately predict outcomes on new data, making it useful for tasks such as fraud detection, medical diagnosis, and recommendation systems. On the other hand, poor generalization results in unreliable predictions, reducing a model's effectiveness.

Good Generalization: The model captures underlying patterns and performs well on both training and unseen data. Example: A spam filter correctly classifies new spam emails based on learned features.

Poor Generalization: The model either memorizes the training data (overfitting) or fails to learn useful patterns (underfitting). Example: A facial recognition system failing to recognize faces in different lighting conditions.

Common Generalization Problems

1. Overfitting

2. Underfitting
3. Selection bias
4. Data Leakage
5. Feature scaling
6. Model complexity

Normalization:

Normalization is the process of rescaling the data so that it has same scale.

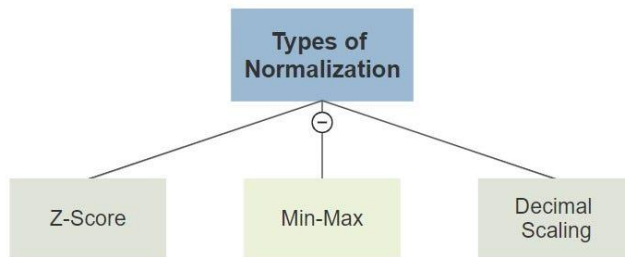
Measurement unit used can affect the data analysis. Hence data are scaled to fall within a smaller range like 0.0 to 1.0. Such transformation or mapping the data to a smaller or common range will help all attributes to gain equal weight. This is known as Normalization.

Normalization is an essential step in the preprocessing of data for machine learning models, and it is a feature scaling technique. Normalization is especially crucial for data manipulation, scaling down, or up the range of data before it is utilized for subsequent stages in the fields of soft computing, cloud computing, etc.

Need for normalization:

1. If the data is not normalized, one feature might completely dominate the others. Normalization makes every data point have the same scale so each feature is equally important.
2. It avoids dependence on the choice of measurement units.
3. The application of data mining algorithms becomes easier, effective and efficient.
4. More specific data analyzing methods can be applied to normalized data.
5. It prevent attributes with initially large ranges (e.g., income) from outweighing attributes with initially smaller ranges (e.g., binary attributes).

Types of Normalization:



Z-Score Normalization

Z-Score Normalization is the process where the features are rescaled so that they have the property of a standard normal distribution with mean(μ) as 0 and standard deviation(σ) as 1.

$$Z = \frac{X - \mu}{\sigma}$$

where:

- X is the data point
- μ is the mean of the attribute values
- σ is the standard deviation of the attribute values

Features:

- It scales the variance at 1.
- It centers the mean at 0.
- It preserves the shape of the original distribution.
- It preserves outliers if they exist.
- Minimum and maximum values vary.

Example 1:

Given the mean and standard deviation for attribute A as 18 and 4.5 respectively. Normalize the value 27 using Z-score normalization.

$$Z = \frac{27 - 18}{4.5}$$

$$Z = 2$$

Example 2:

Z-Score Formula:

$$Z = \frac{X - \mu}{\sigma}$$

Given:

Values: 10, 12, 14, 16, 18

Mean (μ) = 14

Standard Deviation (σ) = 2

Z-Scores:

- $10 \rightarrow (10-14)/2 = -2$
- $12 \rightarrow (12-14)/2 = -1$
- $14 \rightarrow (14-14)/2 = 0$
- $16 \rightarrow (16-14)/2 = 1$
- $18 \rightarrow (18-14)/2 = 2$

Min-Max Normalization

Min-max normalization performs a linear transformation on the original data in range [0, 1] or [-1, 1]. Selecting the target range depends on the nature of data.

If minA and maxA are the minimum and maximum values of an attribute A, Min-max normalization maps a value, vi of A to vi' in the range [new-minA, new-maxA] by computing:

$$v' = \frac{v - \min_A}{\max_A - \min_A} (\text{new_max}_A - \text{new_min}_A) + \text{new_min}_A$$

Min-max normalization preserves the relationships among the original data values. It encounters an “out-of-bounds” error if a future input case for normalization falls outside of the original data range for A.

Features:

- It does not center the mean at 0.
- It makes the variance vary across variables.
- It may not maintain the shape of the original distribution.
- The minimum and maximum values are in the range of [0,1].
- This method is very sensitive to outliers.

Example 1:

Let income range be from 12 to 98. Map income to the range [0.0, 1.0]. By min-max normalization, a value of 73 for income is transformed to?

$$v' = \frac{73 - 12}{98 - 12} (1.0 - 0) + 0$$

$$v' = 0.709$$

Example 2:

Min-Max Normalization Formula:

$$X' = \frac{X - X_{min}}{X_{max} - X_{min}}$$

Given Data:

20, 40, 60, 80, 100

$X_{min} = 20$, $X_{max} = 100$

Normalized Values:

- $20 \rightarrow (20-20)/(100-20) = 0$
- $40 \rightarrow 20/80 = 0.25$
- $60 \rightarrow 40/80 = 0.5$
- $80 \rightarrow 60/80 = 0.75$
- $100 \rightarrow 80/80 = 1$

Decimal Scaling

Decimal Scaling normalizes the value of attribute A by moving the decimal point in the value. This movement of a decimal point depends on the maximum absolute value of the attribute.

$$v' = \frac{v}{10^j}$$

where:

- j is the smallest integer such that $\max(|v_i/10^j|) < 1$

Example 1:

The observed values for attribute A lie in the range from -986 to 917 and the maximum absolute value for attribute A is 986. Normalize the data using Decimal Scaling.

Solution:

Here, to normalize each value of attribute A using decimal scaling, we have to divide each value of attribute A by 1000 i.e. $j=3$.

So, the value -986 would get normalized to -0.986 and 917 would get normalized to 0.917.

Example 2:

Given Data:

120, 450, 980

Max value = 980 $\rightarrow j = 3$

Normalized Values:

- $120 \rightarrow 0.120$
- $450 \rightarrow 0.450$
- $980 \rightarrow 0.980$

Attribute / Feature Selection:

Feature selection is the process of choosing only the most useful input features for a machine learning model. It helps improve model performance, reduces noise and makes results easier to understand.

- Helps remove irrelevant and redundant features
- Improves accuracy and reduces overfitting
- Speeds up model training
- Makes models simpler and easier to interpret



Need of Feature Selection

Feature selection methods are essential in data science and machine learning for several key reasons:

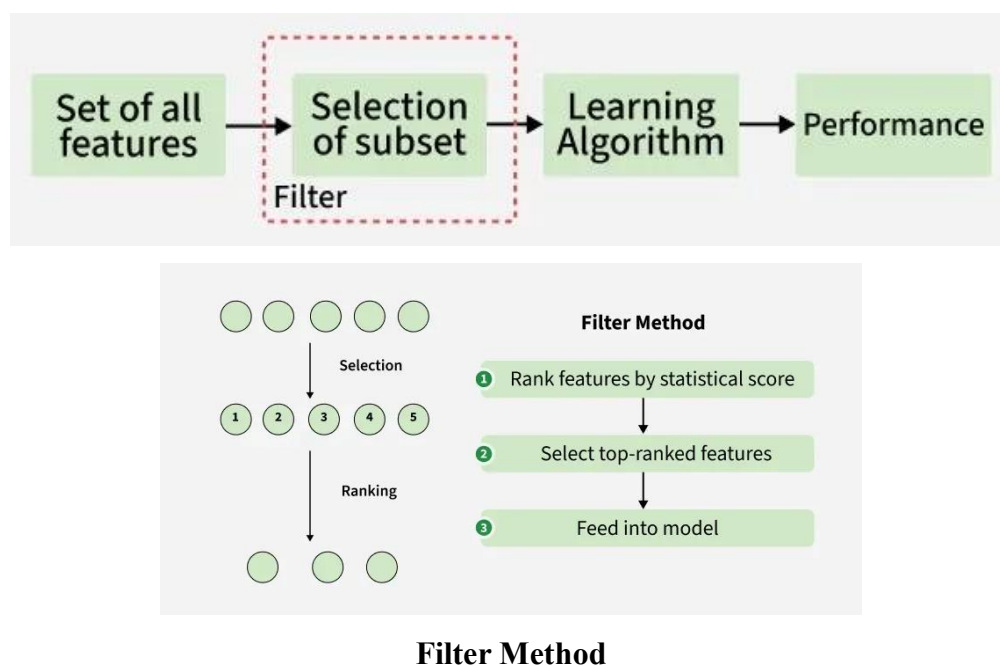
- **Improved Accuracy:** Models learn better when trained on only important features.
- **Faster Training:** Fewer features reduce computation time.
- **Greater Interpretability:** With fewer inputs, understanding model behavior becomes easier.
- **Avoiding the Curse of Dimensionality:** Reduces complexity when working with high-dimensional data.

Types of Feature Selection Methods

There are various algorithms used for feature selection and are grouped into three main categories and each one has its own strengths and trade-offs depending on the use case.

1. Filter Methods

[Filter methods](#) evaluate each feature independently with target variable. Feature with high correlation with target variable are selected as it means this feature has some relation and can help us in making predictions. These methods are used in the preprocessing phase to remove irrelevant or redundant features based on statistical tests (correlation) or other criteria.



Common Filter Techniques

- **[Information Gain](#)**: Measures reduction in entropy when a feature is used.
- **[Chi-square test](#)**: Checks the relationship between categorical features.
- **[Fisher's Score](#)**: Ranks features based on class separability.
- **[Pearson's Correlation Coefficient](#)**: Measures linear relationship between two continuous variables.
- **[Variance Threshold](#)**: Removes features with very low variance.
- **[Mean Absolute Difference](#)**: Similar to variance threshold but uses absolute differences.
- **[Dispersion ratio](#)**: Ratio of arithmetic mean to geometric mean; higher values indicate useful features.

Advantages

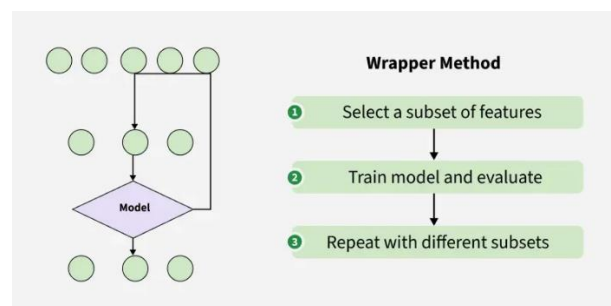
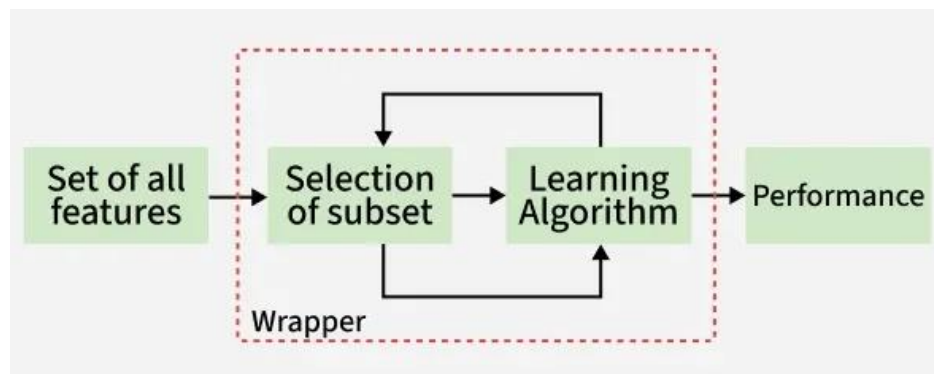
- **Fast and efficient:** Filter methods are computationally inexpensive, making them ideal for large datasets.
- **Easy to implement:** These methods are often built-in to popular machine learning libraries, requiring minimal coding effort.
- **Model Independence:** Filter methods can be used with any type of machine learning model, making them versatile tools.

Limitations

- **Limited interaction with the model:** Since they operate independently, filter methods might miss data interactions that could be important for prediction.
- **Choosing the right metric:** Selecting the appropriate metric for our data and task is important for optimal performance.

2. Wrapper methods

Wrapper methods are also referred as greedy algorithms that train algorithm. They use different combination of features and compute relation between these subset features and target variable and based on conclusion addition and removal of features are done. Stopping criteria for selecting the best subset are usually pre-defined by the person training the model such as when the performance of the model decreases or a specific number of features are achieved.



Wrapper Method

Common Wrapper Techniques

- **Forward Selection**: Start with no features and add one at a time based on improvement.
- **Backward Elimination**: Start with all features and remove the least useful ones.
- **Recursive Feature Elimination (RFE)**: Removes the least important features step by step.

Advantages

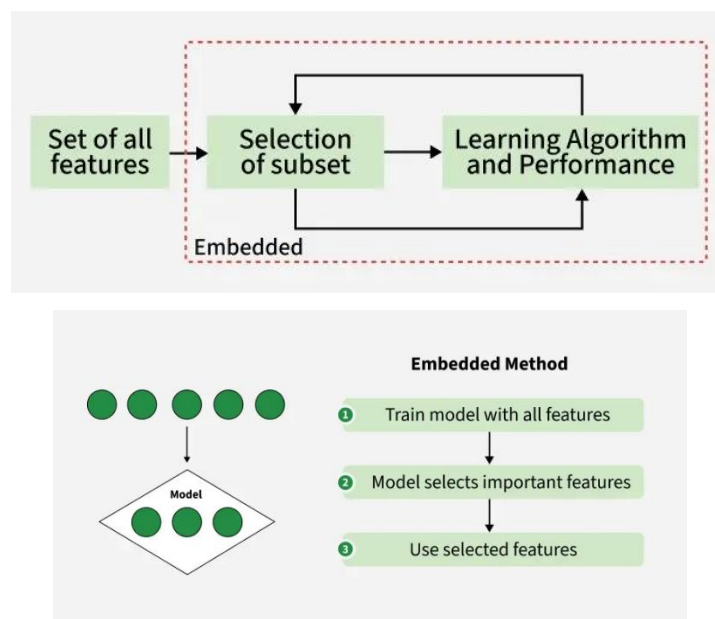
- **Model-specific optimization**: Wrapper methods directly consider how features influence the model, potentially leading to better performance compared to filter methods.
- **Flexible**: These methods can be adapted to various model types and evaluation metrics.

Limitations

- **Computationally expensive**: Evaluating different feature combinations can be time-consuming, especially for large datasets.
- **Risk of overfitting**: Fine-tuning features to a specific model can lead to an overfitted model that performs poorly on unseen data.

3. Embedded methods

Embedded methods perform feature selection during the model training process. They combine the benefits of both filter and wrapper methods. Feature selection is integrated into the model training allowing the model to select the most relevant features based on the training process dynamically.



Embedded Method

Common Embedded Techniques

- **L1 Regularization (Lasso)**: Keeps only features with non-zero coefficients.
- **Decision Trees** and **Random Forests**: Select features based on impurity reduction.
- **Gradient Boosting**: Pick features that reduce prediction error the most

Advantages

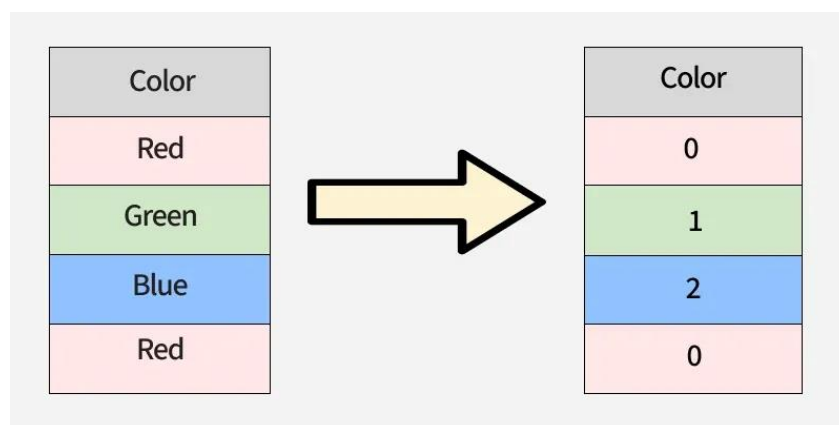
- **Efficient and effective**: Embedded methods can achieve good results without the computational burden of some wrapper methods.
- **Model-specific learning**: Similar to wrapper methods these techniques uses the learning process to identify relevant features.

Limitations

- **Limited interpretability**: Embedded methods can be more challenging to interpret compared to filter methods making it harder to understand why specific features were chosen.
- **Not universally applicable**: Not all machine learning algorithms support embedded feature selection techniques.

Label Encoding:

Label Encoding is a data preprocessing technique in Machine Learning used to convert categorical values into numerical labels. Since most ML algorithms work only with numeric data, categorical features must be encoded before model training. In Label Encoding, each unique category is assigned an integer between 0 and the number of classes.



Label Encoding

The labels are assigned in alphabetical order not based on their position in the dataset which is why encoded values may appear non-sequential when viewed top-down in a DataFrame.

Categorical data is broadly divided into two types:

- **Nominal Data:** Categories without inherent order like colors: red, blue, green.
- **Ordinal Data:** Categories with a natural order like satisfaction levels: low, medium, high.

Label encoding works best for ordinal data, where the assigned numbers reflect the order. But when you apply it to nominal data, the numbers accidentally create a fake ranking (e.g., Red = 0, Blue = 1, Green = 2).

Label Encoding is important because many ML algorithms cannot process string values directly making numerical conversion essential for model training.

- Helps algorithms like SVM, Logistic Regression and KNN work with categorical data.
- More memory efficient compared to One Hot Encoding.
- Suitable when categorical features have a natural order or limited unique classes.
- Ensures consistent and compact representation of categories.

Label Encoding is suitable when converting categorical values into integers will not introduce misleading numeric relationships. It is most useful in the following situations:

- The feature is ordinal and has a natural order.
- The model being used is insensitive to integer ranking such as tree-based algorithms.
- The column contains many unique categories making One-Hot Encoding inefficient.
- Memory efficiency is required and additional dummy columns should be avoided.
- You need a consistent mapping of categories for training, validation and deployment.

Example of label encoding.

Assume that a dataset contains a column called Height with the following elements: tall, medium, and short. We will use label encoding to transform this column from a category to a numerical format. Following the application of label encoding, the Height column is transformed into a numerical column with the elements 0, 1, and 2, where 0 represents the label for long height, 1 represents medium height, and 2 represents short height.

Height	Height
Tall	0
Medium	1
Short	2

One Hot Encoding:

One Hot Encoding is a *method for converting categorical variables into a binary format*. It creates new columns for each category where 1 means the category is present and 0 means it is not. The primary purpose of One Hot Encoding is to ensure that categorical data can be effectively used in machine learning models.

Importance of One Hot Encoding

We use one hot Encoding because:

1. **Eliminating Ordinality:** Many categorical variables have no inherent order (e.g., "Male" and "Female"). If we were to assign numerical values (e.g., Male = 0, Female = 1) the model might mistakenly interpret this as a ranking and lead to biased predictions. One Hot Encoding eliminates this risk by treating each category independently.
2. **Improving Model Performance:** By providing a more detailed representation of categorical variables. One Hot Encoding can help to improve the performance of machine learning models. It allows models to capture complex relationships within the data that might be missed if categorical variables were treated as single entities.
3. **Compatibility with Algorithms:** Many machine learning algorithms particularly based on linear regression and gradient descent which require numerical input. It ensures that categorical variables are converted into a suitable format.

Example:

Imagine we have a dataset with fruits their categorical values and corresponding prices. Using one-hot encoding we can transform these categorical values into numerical form. For example:

- Wherever the fruit is "Apple," the Apple column will have a value of 1 while the other fruit columns (like Mango or Orange) will contain 0.
- This pattern ensures that each categorical value gets its own column represented with binary values (1 or 0) making it usable for machine learning models.

Fruit	Categorical value of fruit	Price
apple	1	5
mango	2	10
apple	1	15
orange	3	20

The output after applying one-hot encoding on the data is given as follows,

Fruit_apple	Fruit_mango	Fruit_orange	price
1	0	0	5
0	1	0	10
1	0	0	15
0	0	1	20

Advantages of Using One Hot Encoding

1. It allows the use of categorical variables in models that require numerical input.
2. It can improve model performance by providing more information to the model about the categorical variable.
3. It can help to avoid the problem of ordinality which can occur when a categorical variable has a natural ordering (e.g. "small", "medium", "large").

Disadvantages of Using One Hot Encoding

1. It can lead to increased dimensionality as a separate column is created for each category in the variable. This can make the model more complex and slow to train.
2. It can lead to sparse data as most observations will have a value of 0 in most of the one-hot encoded columns.

3. It can lead to overfitting especially if there are many categories in the variable and the sample size is relatively small.

Data Reduction

Data reduction is a technique used in data mining to reduce the size of a dataset while still preserving the most important information. This can be beneficial in situations where the dataset is too large to be processed efficiently, or where the dataset contains a large amount of irrelevant or redundant information.

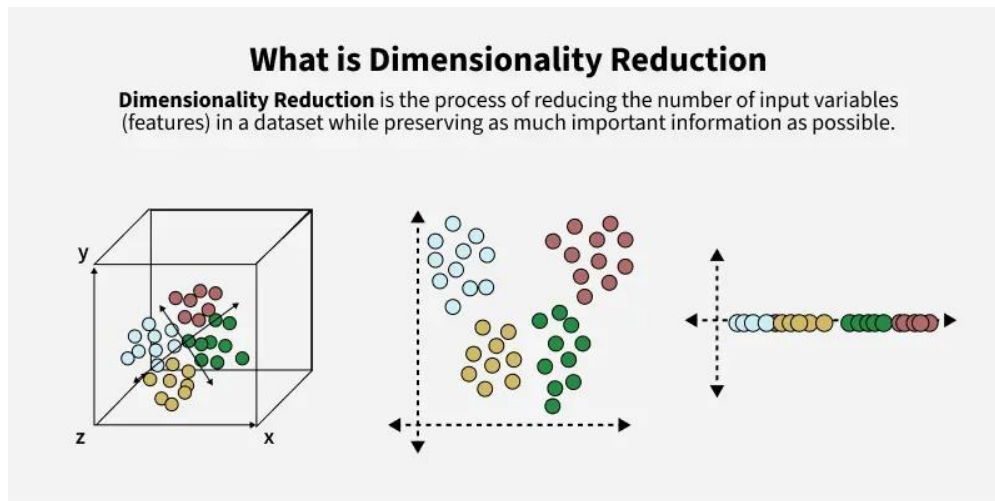
Several Techniques:

1. **Data Sampling:** This technique involves selecting a subset of the data to work with, rather than using the entire dataset. This can be useful for reducing the size of a dataset while still preserving the overall trends and patterns in the data.
2. **Dimensionality Reduction:** This technique involves reducing the number of features in the dataset, either by removing features that are not relevant or by combining multiple features into a single feature.
3. **Data Compression:** This technique involves using techniques such as lossy or lossless compression to reduce the size of a dataset.
4. **Data Discretization:** This technique involves converting continuous data into discrete data by partitioning the range of possible values into intervals or bins.
5. **Feature Selection:** This technique involves selecting a subset of features from the dataset that are most relevant to the task at hand.
6. It's important to note that data reduction can have a trade-off between the accuracy and the size of the data. The more data is reduced, the less accurate the model will be and the less generalizable it will be.

Introduction to Dimensionality Reduction

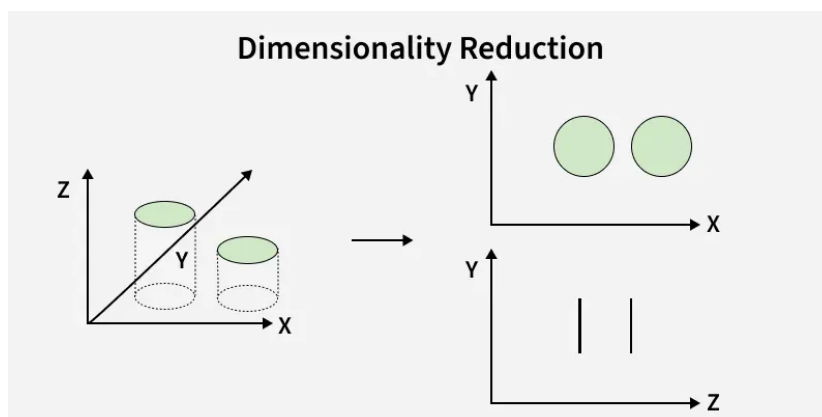
When working with machine learning models, datasets with too many features can cause issues like slow computation and overfitting. Dimensionality reduction helps to reduce the number of features while retaining key information. It converts high-dimensional data into a lower-dimensional space while preserving important details.

For example, when you are building a model to predict house prices with features like bedrooms, square footage and location. If you add too many features such as room condition or flooring type, the dataset becomes large and complex.



How Dimensionality Reduction Works?

Lets understand how dimensionality Reduction is used with the help of example. Imagine a dataset where each data point exists in a 3D space defined by axes X, Y and Z. If most of the data variance occurs along X and Y then the Z-dimension may contribute very little to understanding the structure of the data.



- Before Reduction we can see that data exist in 3D (X,Y,Z). It has high redundancy and Z contributes little meaningful information
- On the right after reducing the dimensionality the data is represented in lower-dimensional spaces. The top plot (X-Y) maintains the meaningful structure while the bottom plot (Z-Y) shows that the Z-dimension contributed little useful information.

This process makes data analysis more efficient hence improving computation speed and visualization while minimizing redundancy.

Dimensionality Reduction Techniques

Dimensionality reduction techniques can be broadly divided into two categories:

1. Feature Selection

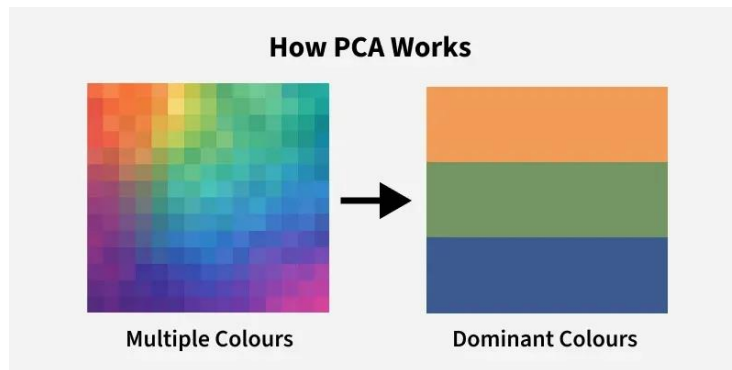
[Feature selection](#) chooses the most relevant features from the dataset without altering them. It helps remove redundant or irrelevant features, improving model efficiency.

2. Feature Extraction

[Feature extraction](#) involves creating new features by combining or transforming the original features. These new features retain most of the dataset's important information in fewer dimensions.

[Principal Component Analysis \(PCA\):](#)

PCA (Principal Component Analysis) is a dimensionality reduction technique and helps us to reduce the number of features in a dataset while keeping the most important information. It changes complex datasets by transforming correlated features into a smaller set of uncorrelated components.



It helps us to remove redundancy, improve computational efficiency and make data easier to visualize and analyze.

How Principal Component Analysis Works

PCA uses linear algebra to transform data into new features called principal components. It finds these by calculating eigenvectors (directions) and eigenvalues (importance) from the covariance matrix. PCA selects the top components with the highest eigenvalues and projects the data onto them simplify the dataset.

PCA helps you find the "most important angles" to view this cloud so you don't miss the big patterns. Here's how it works step by step:

Step 1: Standardize the Data

Different features may have different units and scales like salary vs. age. To compare them fairly PCA first [standardizes](#) the data by making each feature have:

- A mean of 0
- A standard deviation of 1

$$Z = \frac{X - \mu}{\sigma}$$

where:

- μ is the mean of independent features $\mu = \{\mu_1, \mu_2, \dots, \mu_m\}$
- σ is the standard deviation of independent features $\sigma = \{\sigma_1, \sigma_2, \dots, \sigma_m\}$

Step 2: Calculate Covariance Matrix

Next PCA calculates the [covariance matrix](#) to see how features relate to each other whether they increase or decrease together. The covariance between two features x_1 and x_2 is:

$$\text{cov}(x_1, x_2) = \frac{\sum_{i=1}^n (x_{1i} - \bar{x}_1)(x_{2i} - \bar{x}_2)}{n-1}$$

Where:

- $\bar{x}_1$ and $\bar{x}_2$ are the mean values of features x_1 and x_2
- n is the number of data points

The value of covariance can be positive, negative or zeros.

Step 3: Find the Principal Components

PCA identifies new axes where the data spreads out the most:

- **1st Principal Component (PC1):** The direction of maximum variance (most spread).
- **2nd Principal Component (PC2):** The next best direction, *perpendicular to PC1* and so on.

These directions come from the eigenvectors of the covariance matrix and their importance is measured by eigenvalues. For a square matrix A an eigenvector X (a non-zero vector) and its corresponding eigenvalue λ satisfy:

$$AX = \lambda X$$

This means:

- When A acts on X it only stretches or shrinks X by the scalar λ .
- The direction of X remains unchanged hence eigenvectors define "stable directions" of A .

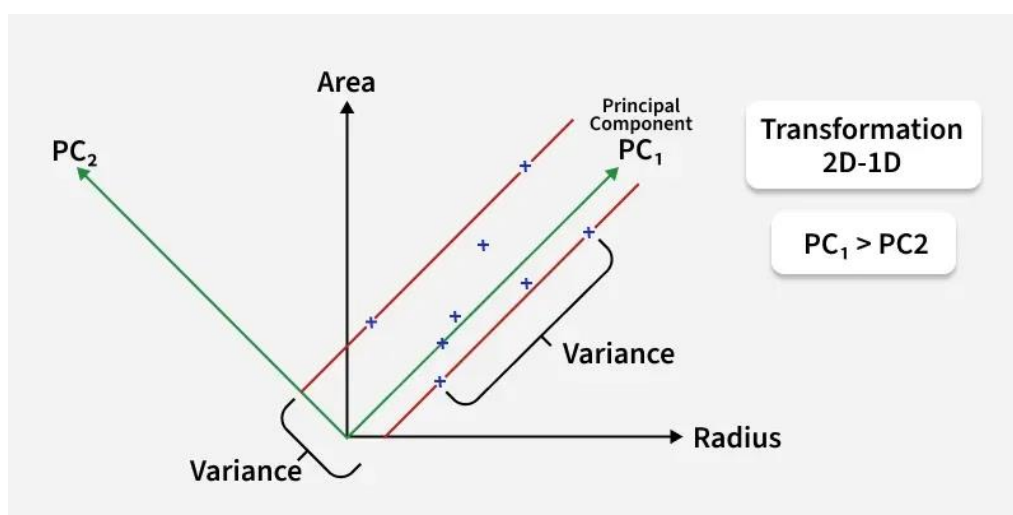
Eigenvalues help rank these directions by importance.

Step 4: Pick the Top Directions & Transform Data

After calculating the eigenvalues and eigenvectors PCA ranks them by the amount of information they capture. We then:

1. Select the top k components that capture most of the variance like 95%.
2. Transform the original dataset by projecting it onto these top components.

This means we reduce the number of features (dimensions) while keeping the important patterns in the data.



Transform this 2D dataset into a 1D representation while preserving as much variance as possible.

In the above image the original dataset has two features "Radius" and "Area" represented by the black axes. PCA identifies two new directions: PC₁ and PC₂ which are the principal components.

- These new axes are rotated versions of the original ones. PC₁ captures the maximum variance in the data meaning it holds the most information while PC₂ captures the remaining variance and is perpendicular to PC₁.

- The spread of data is much wider along PC_1 than along PC_2 . This is why PC_1 is chosen for dimensionality reduction. By projecting the data points (blue crosses) onto PC_1 we effectively transform the 2D data into 1D and retain most of the important structure and patterns.